RetailIQ - Gemini Build Prompt

AI-Powered Retail Intelligence for Inventory Risk Prediction and Local Stock Recovery: prototype blueprint (Next.js, English / Kannada / Hindi)

How to use this document

- Open Gemini (AI Studio, Gemini Code Assist, or any coding-capable Gemini chat). Paste **PART A** first as the master prompt.
- Then paste the build steps in **PART C** one at a time (Step 1, review, Step 2, ...). Building in stages gives far better code than one giant request.
- Attach the report figures (Fig. 3 to Fig. 7 screenshots) to the chat when you paste Part B, because the UI must match them.
- When Gemini asks for it, put the API key in a `.env.local` file. Never paste the key into chat or commit it to Git.

Everything between the START and END markers of each part is meant to be pasted as-is.

PART A - Master prompt (paste first)

```
=== START OF MASTER PROMPT ===

You are a senior full-stack engineer. Build a working PROTOTYPE of "RetailIQ", an
AI-powered retail intelligence and Stock Rescue platform for small neighbourhood
retailers in India. Work step by step, give complete runnable files (no placeholders
like "...rest of code"), and tell me the exact terminal commands to run.

PRODUCT IN ONE LINE
Stock enters -> AI monitors demand and expiry risk -> AI decides (reorder, discount,
bundle) -> AI helps sell it (campaign + local Stock Rescue marketplace) -> sales feed
back into the models.

PROBLEM
Small shops overstock, hold slow movers and lose money on products that expire unsold.
Existing billing apps only record stock. RetailIQ predicts the risk, states the loss in
rupees, recommends the smallest discount or bundle that clears it, and lets nearby
customers reserve the discounted stock and collect it in the shop.

USERS (3 roles)
1. Shopkeeper: full control of ONE shop (products, batches, stock, sales, purchases,
   suppliers, analytics, forecasts, expiry risk, offers, assistant).
2. Customer: browse nearby deals, search, reserve, collect, keep a shopping list.
3. Admin: manages shops/users (keep very simple in the prototype).

HARD RULES (must never be violated)
R1. Every business table has shop_id; every query is scoped to the caller's shop.
    A shopkeeper can never read another shop's data. A customer cannot call
    shopkeeper endpoints.
R2. Nothing past its batch expiry date may ever be offered or shown in the marketplace.
    Offers auto-withdraw before expiry with a category-based margin (dairy/bakery 2
    days, packaged food 7 days, non-food 0 days).
R3. Never sell below purchase price: discount price must be >= purchase price.
R4. The language model NEVER produces business numbers. Prices, quantities, dates and
    shop details come from the database through templates or predefined read-only
    query tools. The LLM only phrases text and explains returned rows. SQL written by
    an LLM is never executed.
R5. Stock entry never writes to the stock table directly. Every intake record is
    validated (Zod), matched to the catalogue and shown to the shopkeeper for
    CONFIRMATION first.
R6. A sale is ONE database transaction: create sale + items, reduce batch stock
    (earliest expiry first, FEFO), update customer history, append to the AI dataset.
R7. Customer location is used only to compute distance. The UI shows distances
    (e.g. "1.2 km"), never raw coordinates.
R8. Every AI recommendation is advice only; the shopkeeper can accept, edit or reject,
    and the choice is stored.

TECH STACK (use exactly this)
- Frontend + app API: Next.js 14+ (App Router), TypeScript, Tailwind CSS, Recharts,
  lucide-react, shadcn/ui primitives, React Hook Form + Zod.
- Auth: Auth.js (NextAuth) with JWT sessions, role + shopId in the token; salted
```

```
  bcrypt password hashes.
- Database: MySQL with Prisma ORM. (If MySQL is hard to set up locally, provide a
  docker-compose.yml for MySQL 8 and a SQLite fallback via an env switch.)
- AI service: Python FastAPI (scikit-learn, statsmodels, xgboost, mlxtend, pandas),
  called by Next.js over REST. Shared secret header between the two services.
- LLM: Google Gemini via the official SDK, key read from GEMINI_API_KEY in .env.local
  (server side only). If the key is missing, fall back to deterministic templates so
  the app still works.
- i18n: next-intl with locale routing /en, /kn, /hi (see Part D).
- Intake: browser barcode scanning (BarcodeDetector API with a zxing fallback),
  typed-sentence entry parsed by Gemini into JSON. Invoice-photo OCR and voice entry
  are Phase 2: create the UI button and a "coming soon" toast only.

EASE OF USE AND SMOOTHNESS REQUIREMENTS
- One-command start: "npm run setup" (install, prisma migrate, seed) and "npm run dev".
  Provide a README with 5-line quick start and a seed script with realistic demo data
  (1 shop, ~48 products, multiple batches, 90 days of sales, a few near-expiry batches,
  3 nearby demo shops for the marketplace).
- Fast, smooth UI: server components for data pages, skeleton loaders, optimistic
  updates for accept/reject actions, debounced search, table pagination, no layout
  shift, toast feedback (sonner), keyboard-friendly forms, mobile responsive.
- Big touch-friendly buttons, plain-language messages, no jargon. Users are shop owners
  who may not be comfortable with English.
- Clear empty states, error states, loading states on every page.
- All AI calls have timeouts and graceful fallbacks; the UI must never freeze.
- Accessibility: labels, focus rings, colour contrast, aria on charts and tables.

=== END OF MASTER PROMPT ===
```

PART B - UI specification (attach Fig. 3 to Fig. 7 screenshots, then paste)

```
=== START OF UI PROMPT ===
The attached screenshots define the visual design. Match them closely: layout,
spacing, card style, colours, icons, table columns and wording. Use the design tokens
and screen specs below.
```

Design tokens

Token	Value
Primary	Green (emerald/green-600 #16A34A, hover #15803D); light tint #F0FDF4
Surface	White cards, 12px radius, 1px #E5E7EB border, very light shadow; page bg #F9FAFB
Status colours	Healthy/Fresh = green, Low stock/Medium risk = amber, High/Critical/Out = red, Info = blue, Slow mover = purple/grey
Font	Inter (next/font). Numbers use tabular-nums
Icons	lucide-react, 18px in nav, 20px in stat-card badges (rounded square tinted background)
Currency	Indian format with rupee sign and lakh grouping, e.g. Rs 1,84,500 (use Intl.NumberFormat 'en-IN' with the rupee symbol in the real UI)

Shopkeeper shell (all shopkeeper pages)

- **Left sidebar** (fixed, white): green shield/box logo + 'RetailIQ' wordmark. Nav items in order: Dashboard, Inventory, Demand Forecasting, Expiry Alerts, Smart Discounts, Stock Rescue, Reports, Settings. Active item = green tint background + green text. At the bottom a small green card 'AI-Powered Inventory Intelligence - Smarter stock. Less waste. Higher profits.' with a leaf/box icon.
- **Top bar**: global search ('Search products, categories, or barcode...' with a Ctrl+K hint), notification bell with red dot, language switcher (EN / ■■■■■ / ■■■■■■), user avatar chip (initial in green circle, name, role 'Shop Owner').
- On mobile the sidebar becomes a slide-over drawer opened by a hamburger button.

Screen 1 - Dashboard (Fig. 3)

- Greeting 'Good morning, {name}!' (time-aware: morning/afternoon/evening) with sub-text 'Here's your store's inventory overview.' and a date-range picker on the right.

- Four stat cards: Total Products (248), Inventory Value (Rs 1,84,500), Low Stock Items (12, amber), Near-Expiry Products (8, red). Each has a small delta line such as '+12% from last month'.
- Row: **Sales Trend** line/area chart (Last 7 days dropdown) and **Top-selling Product Categories** bar chart (units sold).
- Row: **Needs Attention** list (product thumbnail, name, status chip like Low Stock / Expires in 5 days, action button 'Restock' or 'View') with a 'View All' link; **AI Insight** card (e.g. 'Demand for packaged beverages is expected to rise this weekend', with a 'Restock' button); **Recent Activity** feed (stock updated, new product added, expiry alert, low stock alert, with relative times).
- Also show the store health score out of 100 (built from stock accuracy and shares of fast, slow and at-risk stock; weights are constants in one config file), loss avoided and 'biggest opportunity this week'.

Screen 2 - Inventory Management (Fig. 4)

- Title 'Inventory Management', subtitle 'Track products, stock levels, batches, and expiry dates.', green '+ Add Product' button.
- Four stat cards: Total Products (48), Total Stock Value (Rs 68,420), Low Stock Items (5), Near Expiry Products (3).
- Search box plus filters: Category, Stock Status, Expiry Date, and a 'View Filters' control.
- Table 'Products (48)': image, Product, SKU, Category, Stock Quantity, Unit Price, Batch Number, Expiry Date, Stock Status chip (In Stock / Low Stock / Out of Stock), row actions menu. Examples: Milk (1L) Dairy, Bread (Brown) Bakery, Biscuits (Marie) Snacks, Packaged Juice (200ml) Beverages, Rice (5kg) Grains, Cooking Oil (1L) Oils and Fats. Pagination at the bottom (1 2 3 4 5 ... and rows-per-page).
- Add Product opens a drawer with two tabs: 'Product' (name, barcode with a Scan button, category, MRP, purchase price, selling price, min stock, supplier) and 'Add Batch' (batch no, manufacture date, expiry date, quantity). Also an 'Add by sentence' box: e.g. 'add five packets of Parle-G biscuits, each costing five rupees' -> parsed preview -> Confirm.

Screen 3 - Expiry Risk Analysis (Fig. 5)

- Stat cards: Products Near Expiry (8), High Risk Products (3), Estimated Potential Loss (Rs 4,250), Potential Recovery Through Discounts (Rs 2,800), plus a 'View Recommendations' button.
- Left: 'Products at Risk of Expiry' table: Product, Current Stock, Expiry Date, Predicted Unsold Quantity, Risk Level chip (High / Medium / Low), Estimated Loss, Recommended Action (e.g. 'Apply 15% discount'). Right: stacked bar chart 'Predicted Remaining Stock at Expiry' (sold vs leftover per product).
- Bottom: 'AI Recommendations' list with one-click actions: 'Apply 15% discount to eligible products', 'Prioritize products expiring within three days', 'Create bundle offer', 'Reorder smaller quantities'. Each action opens a confirmation sheet showing expected leftover WITH and WITHOUT the discount.

Screen 4 - Demand Forecasting (Fig. 6)

- Stat cards: Forecast Period (7 Days), Products Requiring Reorder (9), Predicted Demand Increase (18%), Potential Stockout Alerts (4).
- Line chart 'Sales vs Predicted Demand' (historical solid line, forecast dashed line with a light confidence band).
- Table 'Reorder Recommendations': Product, Current Stock, Predicted 7-Day Demand, Recommended Reorder Quantity, Stockout Risk chip, Action (Reorder). Button 'Reorder All Needed'. Show days-of-cover per product.
- AI Insight card on the right with a short explanation and 'Key Insights' bullets.

Screen 5 - Smart Discounts, Reports, Settings

- **Smart Discounts**: list of at-risk batches with the recommended discount (smallest 5%-step discount whose expected leftover is acceptable and whose price stays above purchase price), a bundle suggestion when no discount clears the price floor, and a 'Create Offer' button. Offer form: validity dates, quantity, channels (WhatsApp, Stock Rescue). After creation show the campaign pack: poster preview, WhatsApp text with a 'Share on WhatsApp' wa.me link, social caption, and a QR code.
- **Reports**: sales, waste avoided, offers performance (reach, reservations, collection rate), CSV export.
- **Settings**: shop profile, location (browser geolocation), marketplace radius default, language, notification preferences, 'Assistant' toggle.

Screen 6 - Customer app: Stock Rescue marketplace (Fig. 7)

- Separate public layout with a green 'RetailIQ Stock Rescue' header, location chip ('Your Neighbourhood'), search 'Search discounted groceries and daily essentials', nav: Home, Deals Near You, Categories, My Orders, cart and profile icons. Green banner: 'Discover local deals and help reduce food waste'.
- Hero heading 'Fresh Deals Near You' with subtext 'Save money while helping local shops reduce unsold inventory.' Category chips: All, Dairy, Bakery, Beverages, Packaged Foods, Daily Essentials. Radius selector 500 m / 1 km / 2 km.
- Deal cards (5 per row on desktop, 1-2 on mobile): product image, red discount badge (e.g. '25% OFF'), name, shop name, distance ('1.2 km'), discounted price in bold with struck-through original price, 'x left' stock chip, buttons 'Add to Cart' and 'View Deal'. Sorted by great-circle (haversine) distance.
- Reserve flow: choose quantity -> reservation held for a fixed window (e.g. 2 hours) -> 'Show this code/QR at the shop' -> shopkeeper marks collected (recorded as a sale) -> unclaimed reservations auto-release and stock returns. No delivery and no online payment in this version (pay in shop).
- Section 'How Stock Rescue Works' with three numbered steps: 1 Discover Local Deals, 2 Reserve Products, 3 Collect from the Shop. Below it four benefit tiles: Save Money, Support Local Retailers, Reduce Food Waste, A Greener Tomorrow.

```
=== END OF UI PROMPT ===
```

PART C - Build steps (paste one at a time)

Paste each step only after the previous one runs. Ask Gemini to output full files and the exact commands.

Step 1 - Project scaffold and design system

Create the Next.js (App Router, TypeScript, Tailwind) project, install dependencies, set up shadcn/ui, Inter font, design tokens, the shopkeeper shell (sidebar, top bar, mobile drawer), and next-intl with /en, /kn, /hi routing and a language switcher. Provide the folder structure first, then all files. Include .env.example with DATABASE_URL, AUTH_SECRET, GEMINI_API_KEY, AI_SERVICE_URL and AI_SERVICE_KEY.

Step 2 - Database, auth and roles

Write the Prisma schema (tables below), migrations, Auth.js credentials login/register, role + shopId in JWT, route protection middleware, and a helper 'requireShop()' that every server action and API route must use so that all queries are scoped by shop_id. Add a seed script with the demo data described in Part A.

Step 3 - Core retail system (the non-AI baseline)

Build Inventory page and drawer, products/batches CRUD, purchases (adds a batch, updates supplier and cost), sales with FEFO batch deduction inside one Prisma transaction, suppliers, customers, and the Dashboard with real numbers from the database. Include barcode scan and the typed-sentence intake with the confirm step. The app must be a fully working retail manager at this point.

Step 4 - AI service and analytics pages

Create the FastAPI service: /forecast, /reorder, /classify, /expiry-risk, /discount, /rules. Implement the logic in Part E. Wire the Demand Forecasting, Expiry Risk and Smart Discounts pages to it. If the AI service is down, show baseline (moving-average and calendar-class) results with a small 'basic mode' badge.

Step 5 - Offers, campaigns and the assistant

Offer creation, campaign generator (template + Gemini wording only; numbers inserted from the offer record; shopkeeper approves before publish), poster preview, wa.me share link, QR code. Build the assistant chat with intent detection and the read-only tool list in Part E; the LLM only explains returned rows.

Step 6 - Customer app and Stock Rescue marketplace

Public customer layout, nearby deals by haversine distance, radius filter, cart/reserve flow with time-limited holds, QR/code collection by the shopkeeper, auto-release job (cron route), and auto-withdrawal of offers before expiry. Show distances only.

Step 7 - Polish, tests and README

Skeletons, empty/error states, toasts, responsive checks, accessibility pass, Playwright smoke tests for login, add stock, make a sale, create an offer, reserve and collect; unit tests for FEFO, price-floor and expiry-withdrawal rules. Final README with setup, env vars, demo accounts and how to add a new language.

PART D - Multi-language support (English, Kannada, Hindi)

```
=== START OF I18N PROMPT ===
Implement internationalisation with next-intl.
- Locales: en (default), kn (Kannada), hi (Hindi). Routes /en/..., /kn/..., /hi/...
- All UI text lives in messages/en.json, messages/kn.json, messages/hi.json. NO
  hard-coded strings in components. Use namespaced keys (nav, dashboard, inventory,
  expiry, forecast, discounts, rescue, assistant, common, errors).
- You (Gemini) must write complete, natural Kannada and Hindi translations for every
  key. Use simple everyday shop vocabulary, not literal word-for-word translations.
  Keep brand name "RetailIQ" and units (kg, L, ml) unchanged.
- Add a language switcher in the top bar and customer header; remember the choice in
  a cookie and in the user profile.
- Fonts: load Noto Sans Kannada and Noto Sans Devanagari with next/font and apply
  them per locale so glyphs render correctly everywhere, including Recharts labels.
- Numbers, currency and dates: use Intl with en-IN, kn-IN, hi-IN. Keep Latin digits
  by default (setting to switch to native digits is optional).
- Pluralisation and ICU placeholders: use ICU message format, e.g.
  "{count} products expire this week".
- AI-written text (campaign wording, assistant answers) must be generated in the
  user's current locale: pass the locale in the Gemini prompt. Numbers still come from
  the database, never from the model.
- Typed stock entry and the assistant must accept input in all three languages
  (e.g. Kannada or Hindi sentences) and map product names to the catalogue.
- Provide a script "npm run i18n:check" that fails if any key is missing in any
  locale, and document how to add another language (e.g. Telugu, Tamil) later.
- Make sure long Kannada/Hindi strings do not break layouts: use flexible widths,
  wrap text, and test all screens in all three languages.
=== END OF I18N PROMPT ===
```

PART E - Data model and AI logic (paste with Step 2 and Step 4)

Database tables (Prisma, all business tables carry shop_id)

Table	Key fields
users	id, name, email, phone, password_hash, role (ADMIN/SHOPKEEPER/CUSTOMER), shop_id (nullable for customers), preferred_locale
shops	id, name, owner_id, address, lat, lng, default_radius_m, created_at
products	id, shop_id, name, barcode, sku, category, mrp, purchase_price, selling_price, min_stock, reorder_level, supplier_id, shelf_life_days, image_url
batches	id, shop_id, product_id, batch_no, mfg_date, expiry_date, qty_received, qty_on_hand, cost_price
inventory	product_id, shop_id, total_qty (derived/maintained inside transactions)
sales / sale_items	sales(id, shop_id, customer_id, total, source: POS or RESERVATION, created_at); sale_items(sale_id, product_id, batch_id, qty, unit_price)
purchases / purchase_items	purchases(id, shop_id, supplier_id, ordered_at, received_at); purchase_items(purchase_id, product_id, batch_no, qty, cost, expiry_date)
suppliers	id, shop_id, name, phone, lead_time_days (estimated from history), pack_size
customers	id, shop_id, name, phone, last_purchase_at (collect only what features need)
offers	id, shop_id, batch_id, type (DISCOUNT/BUNDLE), discount_pct, offer_price, qty, valid_from, valid_to, channels, status (DRAFT/PUBLISHED/WITHDRAWN/EXPIRED), withdraw_at, recommended_by_ai
reservations	id, offer_id, customer_id, qty, code, hold_until, status (HELD/COLLECTED/RELEASED)
forecasts	id, shop_id, product_id, horizon_days, units, model_name, model_version, created_at

Table	Key fields
recommendations	id, shop_id, kind, payload_json, status (PENDING/ACCEPTED/EDITED/REJECTED), created_at

AI service logic (FastAPI)

- **Demand forecasting**: build a daily series per product from sale_items, inserting ZERO on days with no sales. Features: lags 1/7/14/28, rolling means 7/28, day of week, day of month, holiday/festival flag, price. Try in order: moving average (baseline) -> exponential smoothing -> ARIMA -> random forest -> XGBoost. Pick per product with rolling-origin validation (never random split). Products with under ~4 weeks of history use the moving average; very slow items are forecast as weekly totals. Return 7-day and 30-day units and MASE vs naive.
- **Reorder**: suggested = forecast over cover period + safety stock - on hand - on order. Example: 100 + 20 - 30 = 90. Round up to supplier pack size; cap cover period at shelf life. Also return days of cover = stock / daily demand. Advice only; store accept/edit/reject.
- **Mover classification**: share of days sold, average daily sales, days since last sale, revenue share. Percentile cut-offs -> fast / medium / slow. Dead stock = unsold beyond a per-category limit. Final states: fresh, normal, slow-moving, near-expiry, dead stock.
- **Expiry risk**: calendar classes (critical <= 3 days, high <= 7, moderate <= 30, healthy beyond). Then project leftover = on_hand - forecast_daily x days_left (floor 0), loss = leftover x purchase_price. Move a batch up one class if leftover share exceeds a tolerance. Allocate forecast demand to the oldest batch first. Message example: '40 packets at risk, estimated loss Rs 400 if unsold'.
- **Discount and bundle**: grid-search discounts in 5% steps; keep the smallest whose expected leftover is acceptable and whose price >= purchase price. Use a cautious default price elasticity and update it after each completed offer. If nothing clears the floor, report remaining loss and suggest a bundle (association rules + category pairing) or Stock Rescue. Show expected leftover with and without the discount.
- **Recommendations**: Apriori/FP-growth on past baskets; keep rules with lift > 1 and sufficient support. Used for bundle candidates and product-page suggestions.
- **Model lifecycle**: weekly retrain and whenever a product's recent error climbs well above its norm. Store model_version with every forecast. Compare projected vs actual leftover to monitor the expiry model.

Assistant: predefined read-only tools (the LLM may only choose among these)

Intent	Tool (parameterised, shop-scoped, read only)
Which products will run out soon?	getLowStockForecast(shopId, days)
Which batches are close to expiry?	getExpiringBatches(shopId, days)
What sold the most this week?	getTopSellers(shopId, from, to)
What should I buy tomorrow?	getReorderSuggestions(shopId)
How much stock is at risk?	getAtRiskValue(shopId)

Flow: detect intent -> select tool -> run it -> give the returned rows to Gemini to explain in 2-3 sentences in the user's language. Gemini may not introduce any figure absent from the rows. After generation, verify every number in the answer appears in the tool result; otherwise replace the answer with a plain table of the rows.

PART F - Scope, acceptance checks and evaluation

Prototype scope (MVP = Phases 1 to 5, plus a basic Phase 6)

- Included: auth and roles, core retail system, analytics dashboard, forecasting, reorder, classification, expiry risk, discount and bundle recommendation, campaigns, assistant, basic customer marketplace with reserve and collect.
- Later: invoice-photo OCR, voice entry, price optimisation, multi-store, automated procurement, collaborative filtering, delivery, regional languages beyond Kannada and Hindi.

Acceptance checklist (ask Gemini to confirm each)

#	Check
1	A sale reduces earliest-expiry batch first and is atomic; stock never goes negative.
2	Shop A user cannot read or change shop B data through any route, including the assistant.
3	No offer or marketplace card ever shows a batch past its withdraw_at/expiry.
4	Recommended discount price is never below purchase price.
5	Offer text, prices and quantities match the database exactly (template-inserted).
6	Barcode and typed intake always show a confirmation step before saving.
7	All screens render correctly in English, Kannada and Hindi; i18n:check passes.
8	App still works (basic mode) when the AI service or Gemini key is unavailable.
9	Dashboard, Inventory, Expiry, Forecasting and Marketplace visually match Fig. 3 to Fig. 7.

Evaluation hooks to build in (from the report)

- Forecast metrics: MAE, RMSE, MASE (target MASE below 1 vs naive), time-ordered validation only.
- Expiry model: precision and recall of at-risk flags (target recall >= 80%) and leftover error.
- Discount replay: compare no action vs flat discount vs recommender on stock written off, revenue recovered, discount depth and margin retained.
- Marketplace: reach and share of reservations collected. Assistant: intent accuracy and numeric faithfulness.
- Usability: entry time versus form-based entry and a System Usability Scale questionnaire.
- Provide a synthetic data generator (weekly patterns, festival peaks, intermittent items, batches with shelf life and expiry) so these experiments can run without real shop data.

Output format Gemini should follow every time

- Start with the file tree for the step, then give every file in full, each in its own code block with its path.
- End with: exact commands to run, what I should see in the browser, and 3 quick tests to verify the step.
- If something is ambiguous, make the simplest reasonable assumption, state it in one line and continue.